WASTE IMAGE CLASSIFICATION

A mini project report

Submitted by

SIDDHARTH KP (241501203)

SHANE A (241501197)

YESWANTH T (241501253)

AI23531 - DEEP LEARNING

in

Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

(Autonomous)

Regulations 2023

BONAFIDE CERTIFICATE

Certified that this the bonafide record of work carried out by SIDDHARTH KP (241501203), SHANE A (241501197), YESWANTH T (241501253) in the mini project titled “Waste Image Classification” as part of the Deep Learning course during the academic year 2026-2027.

TABLE OF CONTENTS

S. No	Section / Chapter Title	Page No
	Abstract	4
1	**Chapter 1: Introduction**	5
	1.1 Problem Statement and Explanation	5
	1.2 Literature Review	5
	1.3 Existing System	6
	1.4 Proposed System	6
2	**Chapter 2: Data Collection and Preprocessing**	7
	2.1 Data Collection – About the Dataset	7
	2.2 Architecture Diagram and Workflow Explanation	7
	2.3 Data Preprocessing	9
3	**Chapter 3: Model Performance**	10
	3.1 Exploratory Data Analysis (EDA)	10
	3.2 Model Performance Results	10
	3.3 Summary	11
4	**Chapter 4: Results and Discussion**	12
	4.1 Overall Performance Comparison	12
	4.2 Statistical Significance and Repeated Runs	12
	4.3 Model Complexity Analysis	13
	4.4 Misclassification and Robustness Analysis	13
	4.5 Prediction Demonstration Analysis	13
	4.6 Discussion	14
	4.7 Conclusion	14
5	**References**	15

ABSTRACT

Waste Image Classification is a deep learning-based mini project that automatically identifies the category of a waste item from a photograph. Manual waste segregation is slow, inconsistent and difficult to scale, so the project applies computer vision to classify an image into one of six categories: cardboard, glass, metal, paper, plastic and trash. The system is implemented in Python using TensorFlow and Keras and runs in Google Colab with optional GPU acceleration.

Two deep learning models are trained and compared. The baseline is a Convolutional Neural Network (CNN) with three convolution and max-pooling blocks trained from scratch on the waste images. The advanced model uses transfer learning with MobileNetV2 pretrained on ImageNet: a new classification head is first trained while the pretrained base is frozen, and the last 20 layers of the base are then fine-tuned with a very small learning rate. The dataset is divided into approximately 70% training, 15% validation and 15% test data, and data augmentation (random flip, rotation and zoom), dropout and early stopping are used to reduce overfitting.

Performance is measured on the unseen test set using accuracy, macro-averaged precision, recall and F1-score, a classification report and confusion matrices, which together show not only how often the models are correct but also which waste categories they confuse. The trained model can also predict the category and confidence score of a newly uploaded image and can be saved in the .keras format for reuse. The project demonstrates a practical application of deep learning to environmental sustainability and shows how transfer learning can be applied to a relatively small image dataset.

Keywords: Waste Classification, Deep Learning, Convolutional Neural Network, Transfer Learning, MobileNetV2, TrashNet, Image Classification, TensorFlow, Keras, Google Colab.

CHAPTER 1: INTRODUCTION

1.1 Problem Statement and Explanation

Rapid urbanization has increased the volume of solid waste, and effective recycling depends on separating materials such as paper, glass, metal and plastic at the source. Manual sorting is time-consuming, depends on human attention and can lead to incorrect segregation, which contaminates recyclable material and increases landfill use. Items from different categories can also look similar: a transparent plastic bottle may resemble glass, and crumpled paper may resemble cardboard.

The problem addressed by this project is to develop a deep learning system that accepts an image of a waste item and predicts its category together with a confidence score. The system should learn visual features such as colour, texture, shape and surface patterns automatically from labelled images instead of relying on hand-written rules, should be evaluated on images that were not used for training, and should be easy to run in a Google Colab notebook.

Project Objectives

- To collect and inspect a labelled six-class waste image dataset.
- To preprocess the images and divide them into training, validation and test sets.
- To build and train a baseline CNN from scratch.
- To build a MobileNetV2 transfer-learning model and fine-tune selected layers.
- To evaluate both models using accuracy, precision, recall, F1-score and confusion matrices.
- To compare the models, analyse the most frequently confused classes and predict the category of a new image.
- To save the trained models for future inference.

1.2 Literature Review

Automated waste sorting has been studied as an image classification problem. Yang and Thung created the TrashNet dataset, which contains photographs of individual waste items placed on a plain background and labelled as cardboard, glass, metal, paper, plastic or trash. They compared classical machine learning with a convolutional neural network and showed that image-based recognition of recyclable material is feasible. TrashNet has since become a common benchmark for student and research projects on waste classification.

Convolutional neural networks are the standard approach to image recognition because stacked convolution layers learn a hierarchy of features, from edges and textures in early layers to object parts and shapes in deeper layers (LeCun, Bengio and Hinton, 2015). However, training a deep network from scratch normally requires a large amount of labelled data. Transfer learning addresses this by reusing a network pretrained on a large dataset such as ImageNet and adapting it to a new task, which is particularly useful when the target dataset contains only a few thousand images.

MobileNetV2 (Sandler et al., 2018) is an efficient architecture built on depthwise-separable convolutions and inverted residual blocks with linear bottlenecks. It provides strong ImageNet features with a comparatively small number of parameters, which makes it practical for Google Colab and for later deployment on mobile or embedded devices. This mini project therefore compares a small custom CNN with a MobileNetV2 model that uses transfer learning and fine-tuning. A limitation noted in the literature is that TrashNet images are taken against controlled backgrounds, so models trained on it may not generalize directly to cluttered real-world scenes.

1.3 Existing System

In most places waste segregation is still performed manually by households, sanitation workers or sorting-facility staff. Some industrial facilities use sensor-based sorting such as magnetic separation for metals, optical or near-infrared sensors for plastics and air-jet separation, which are effective but expensive and require specialised equipment. Simple rule-based or classical computer vision methods that use hand-crafted colour or shape features are inexpensive but are sensitive to lighting, orientation and background.

The common limitations of existing approaches are the dependence on human effort and consistency, high equipment cost, limited ability to distinguish visually similar materials, and poor adaptability to new item types. Training a deep network from scratch for this task can also overfit when the available dataset is small.

1.4 Proposed System

The proposed system is an image-based waste classifier built with TensorFlow and Keras. Images are resized to 160 × 160 pixels and loaded in batches of 32. The dataset is divided into approximately 70% training, 15% validation and 15% test data using a fixed random seed of 42 for reproducibility. Random horizontal flip, rotation and zoom are applied as data augmentation during training.

Two models are developed. The baseline CNN contains three convolution and max-pooling blocks with 32, 64 and 128 filters, global average pooling, a 128-unit dense layer with 40% dropout and a six-unit softmax output, and is trained from scratch with the Adam optimizer and sparse categorical cross-entropy loss. The advanced model uses MobileNetV2 with ImageNet weights and a new head consisting of global average pooling, 30% dropout and a six-unit softmax layer. It is trained in two stages: the pretrained base is first frozen (learning rate 0.001), and the last 20 layers are then unfrozen and fine-tuned with a learning rate of 0.00001. Early stopping on validation loss is used to limit overfitting.

Both models are evaluated on the held-out test set using accuracy, macro-averaged precision, recall and F1-score and confusion matrices. The system can then predict the category and confidence of a newly uploaded image. The outcome of the comparison depends on the data, training and hardware, so the measured test results, rather than an assumed ranking, are used to identify the better model.

CHAPTER 2: DATA COLLECTION AND PREPROCESSING

2.1 Data Collection – About the Dataset

The project uses the TrashNet waste image dataset, a publicly available collection of photographs of individual waste items. The dataset is downloaded programmatically in the notebook from a public GitHub repository containing the resized TrashNet images, so no manual upload is required. The images are organised in one folder per class, and the notebook locates the six class folders automatically. The standard TrashNet distribution contains 2,527 images in six classes.

Class	Description	Images
Cardboard	Boxes and cardboard packaging	403
Glass	Glass bottles and containers	501
Metal	Metal cans and objects	410
Paper	Paper and paper products	594
Plastic	Plastic bottles and packaging	482
Trash	Other non-recyclable or mixed waste	137
Total		**2,527**

The class counts above are those of the standard TrashNet release; the exact numbers in the downloaded copy are printed by the dataset-loading cell of the notebook. The “trash” class is considerably smaller than the others, which creates a class imbalance that is discussed in Chapter 3. TrashNet images are photographed against a plain background under controlled lighting, which makes the dataset convenient for learning but may limit performance on cluttered real-world images. The dataset should be used in accordance with its published licence and cited as the source.

2.2 Architecture Diagram and Workflow Explanation

Figure 1 shows the main workflow. The labelled waste images are loaded and preprocessed, divided into training, validation and test sets, and augmented. The same data are used to train the baseline CNN from scratch and the MobileNetV2 transfer-learning model. Both models are then evaluated on the unseen test set, compared, and the better model is used for prediction on new images.

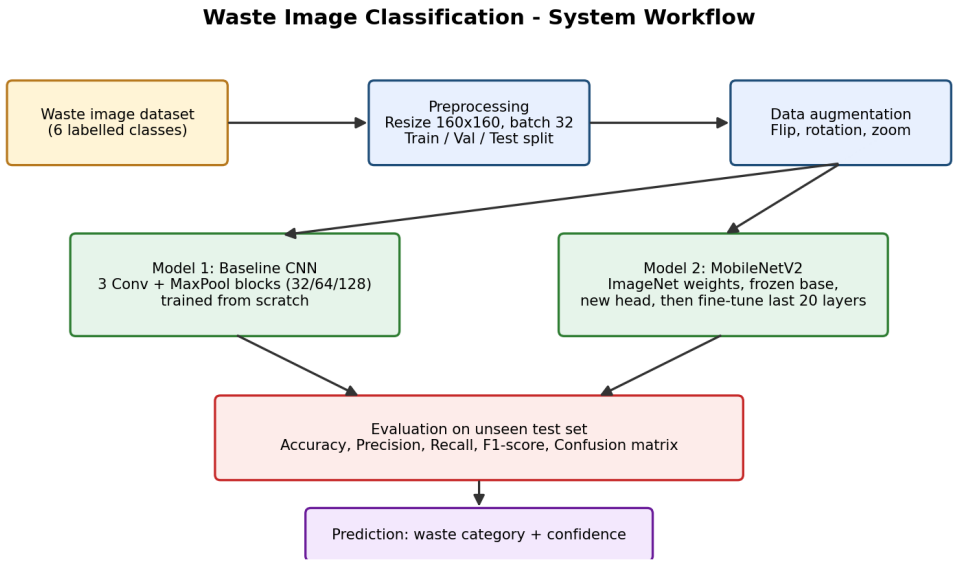


Figure 1. System workflow of Waste Image Classification.

Workflow Steps

- **Dataset loading:** download and extract the TrashNet images and locate the six class folders.
- **Exploration:** count images per class, plot the class distribution and display sample images.
- **Preprocessing:** resize images to 160 × 160, create batches of 32 and split into training, validation and test sets.
- **Augmentation:** apply random flip, rotation and zoom during training.
- **Model 1:** build and train the baseline CNN from scratch.
- **Model 2:** train a new head on frozen MobileNetV2 features, then fine-tune the last 20 layers.
- **Evaluation:** compute accuracy, precision, recall, F1-score, classification reports and confusion matrices.
- **Prediction and saving:** classify an uploaded image and save both trained models in .keras format.

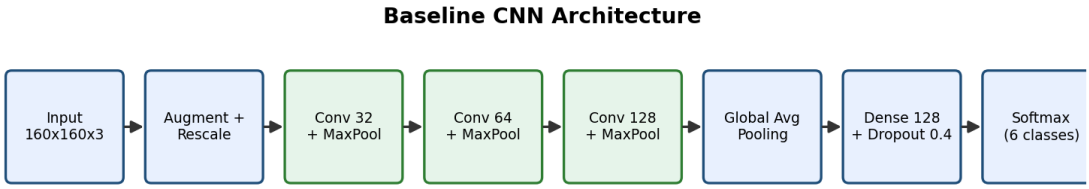


Figure 2. Architecture of the baseline CNN.

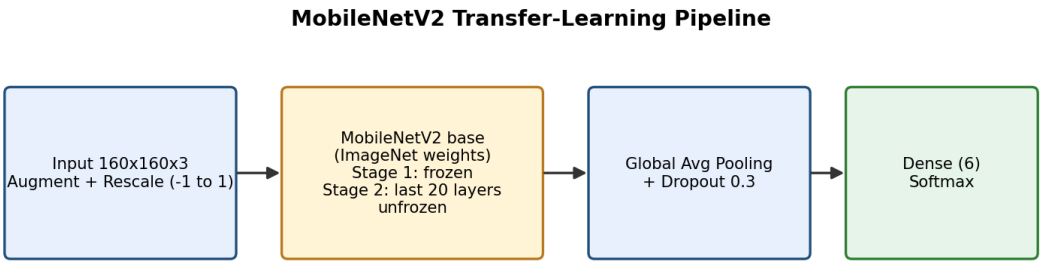


Figure 3. MobileNetV2 transfer-learning and fine-tuning pipeline.

2.3 Data Preprocessing

Images are loaded with the Keras image_dataset_from_directory utility, which reads the class folders, assigns integer labels in alphabetical order (cardboard, glass, metal, paper, plastic, trash) and resizes every image to 160 × 160 pixels with three colour channels. Images are processed in batches of 32, and prefetching is used so that the next batch is prepared while the model is processing the current one.

The data are divided using a fixed seed of 42. About 70% of the images form the training set. The remaining 30% are loaded separately and divided by batches into a validation set (about 15%), used to monitor learning, and a test set (about 15%), used only for the final evaluation. For a standard 2,527-image copy this corresponds to roughly 1,769 training images and about 758 images shared between validation and test. Because this remaining subset is divided by batches, it must be loaded with shuffling enabled; otherwise the images would be separated in class-folder order and the validation and test sets would contain different classes. The notebook therefore sets shuffle=True for this subset, and the exact partition sizes and class proportions should be verified.

Pixel values are scaled inside the models. The baseline CNN rescales pixels from 0–255 to 0–1, while the MobileNetV2 model rescales them to the range −1 to +1 expected by its pretrained weights. Because scaling is part of each model, a new image only needs to be resized to 160 × 160 before prediction. Data augmentation layers (random horizontal flip, rotation of up to 10% of a full turn and zoom of up to 10%) are active only during training and increase the variety of the small training set.

CHAPTER 3: MODEL PERFORMANCE

3.1 Exploratory Data Analysis (EDA)

Exploratory analysis checks the dataset before training. The number of images in each class is counted and plotted as a bar chart (Figure 4), and a grid of sample images with their labels is displayed to confirm that images load correctly and that the labels match the content.

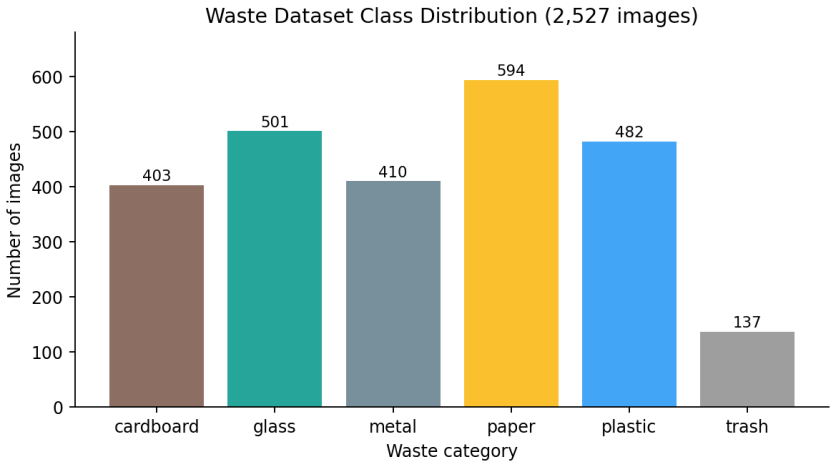


Figure 4. Class distribution of the waste image dataset.

The distribution shows that paper and glass are the largest classes while trash is much smaller, with roughly one quarter of the images of the paper class. With such imbalance a model can achieve a high overall accuracy while performing poorly on the small class. For this reason macro-averaged precision, recall and F1-score, which weight all six classes equally, are reported together with accuracy, and class-wise results are examined in the classification report and confusion matrix.

Additional useful checks include image dimensions and file formats, brightness and colour distribution, and whether items such as transparent plastic and glass or cardboard and paper look visually similar, since these are the pairs most likely to be confused.

3.2 Model Performance Results

Both models are trained on the training set, monitored on the validation set and evaluated once on the unseen test set. The training configuration is summarised below.

Setting	**Baseline CNN**	**MobileNetV2**
Input size	160 × 160 × 3	160 × 160 × 3
Batch size	32	32
Initial weights	Random (from scratch)	ImageNet pretrained
Optimizer / learning rate	Adam (default)	Adam, 0.001 then 0.00001

Setting	Baseline CNN	MobileNetV2
Epochs (maximum)	8	5 (head) + 3 (fine-tuning)
Regularization	Augmentation, dropout 0.4, early stopping	Augmentation, dropout 0.3, early stopping
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy

The evaluation metrics are defined as follows. **Accuracy** is the proportion of correctly classified test images. **Precision** is the fraction of images predicted as a class that truly belong to it, **recall** is the fraction of images of a class that are correctly found, and **F1-score** is the harmonic mean of precision and recall. Macro averaging computes each metric per class and averages the six values equally. The **confusion matrix** shows how many images of each true class were assigned to each predicted class and therefore reveals which categories are confused.

Performance Results Table (test set, from the Colab Cell 10 output)

Model	Accuracy	Precision (macro)	Recall (macro)	F1-score (macro)
Baseline CNN	_____	_____	_____	_____
MobileNetV2 (fine-tuned)	_____	_____	_____	_____

The blank cells are to be filled with the measured values printed by the evaluation cell of the notebook. No result is assumed in advance because the scores depend on the dataset copy, the random split, the training process and the hardware. Figures 5 to 7 should be taken from the same run.

Insert Colab output here
Figure 5: training and validation accuracy / loss curves (Cell 9)

Figure 5. Training and validation accuracy and loss of the CNN and MobileNetV2 (initial training stages).

3.3 Summary

The project trains a baseline CNN from scratch and a MobileNetV2 transfer-learning model, which is first trained with a frozen base and then fine-tuned. The baseline shows what a small custom network can learn from the available images, while MobileNetV2 reuses features learned from ImageNet and adapts only a small head and the final layers. The comparison should consider overall accuracy together with macro precision, recall and F1-score and the class-wise errors, rather than accuracy alone.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Overall Performance Comparison

The two models are compared on the same unseen test set. The comparison discusses which model achieves higher accuracy, precision, recall and F1-score, whether the gap between training and validation performance suggests overfitting, and whether the additional fine-tuning stage gives a meaningful improvement over training only the new head. The bar chart produced by the comparison cell of the notebook is inserted here after the experiment is run.

Insert Colab output here
Figure 6: model comparison bar chart (Cell 11)

Figure 6. Performance comparison of the baseline CNN and MobileNetV2 on the test set.

Insert Colab output here
Figure 7: confusion matrices of both models (Cell 10)

Figure 7. Confusion matrices of the baseline CNN and MobileNetV2 on the test set.

4.2 Statistical Significance and Repeated Runs

Neural network training is stochastic: results can change with the random split, weight initialization, augmentation and the order of batches. The test set also contains only a few hundred images, and its small classes such as trash contain only a few dozen, so a small number of mistakes can change class-wise scores noticeably. To make conclusions more reliable, training should be repeated with

different random seeds or with k-fold cross-validation and the mean and standard deviation of accuracy and macro-F1 reported. A single run on one split is a useful demonstration but is not sufficient to support a broad statistical claim that one model is always better than the other.

4.3 Model Complexity Analysis

The baseline CNN is small. Its three convolution layers contain 896, 18,496 and 73,856 parameters, the dense layer 16,512 and the output layer 774, giving approximately 110,500 trainable parameters, all learned from scratch. The MobileNetV2 base contains about 2.26 million parameters, but in the first stage these are frozen and only the new six-class head, with 7,686 parameters, is trained. During fine-tuning the last 20 layers of the base are also updated using a very small learning rate of 0.00001 so that the pretrained features are adjusted gently instead of being overwritten.

Although MobileNetV2 has more parameters in total, it reuses features learned from a very large dataset and is designed to be computationally efficient. Training time depends on the GPU, image size and number of epochs and should be recorded from the Colab run. The baseline is expected to be cheaper per epoch because it is smaller, but this should be confirmed by measurement.

4.4 Misclassification and Robustness Analysis

The confusion matrix and classification report show which classes are most frequently misclassified. Pairs likely to cause errors in waste images are glass and plastic (transparent, reflective containers), paper and cardboard (similar colour and texture), and metal and glass (reflective surfaces). The trash class is the smallest and contains very mixed objects, so it is often the hardest to recognise. These observations should be confirmed against the actual confusion matrices of the final run, and the most common confusions recorded here.

Robustness depends on how well the model generalizes beyond the dataset. TrashNet photographs are taken on a plain background under controlled lighting, so a model may perform worse on photographs with cluttered backgrounds, several objects, crushed or dirty items, shadows or unusual viewpoints. Data augmentation, dropout and early stopping reduce overfitting but do not remove this domain gap. Collecting more varied images and rebalancing the classes would be expected to improve real-world performance.

4.5 Prediction Demonstration Analysis

After training, the notebook demonstrates the model on a newly uploaded image. The image is converted to RGB, resized to 160 × 160 and passed to the trained MobileNetV2 model, which returns six class probabilities. The class with the highest probability is shown as the predicted waste category together with its confidence. This step does not retrain the model.

The confidence is the model’s predicted probability for the selected class; it is not a guarantee that the prediction is correct, and an image from a category or background unlike the training data may still receive a high confidence for a wrong class.

Insert Colab output here

Figure 8: uploaded test image with predicted class and confidence (Cell 12)

Figure 8. Prediction of the trained model on a newly uploaded waste image.

4.6 Discussion

The project is appropriate for an educational mini project because the pipeline is transparent and each stage can be inspected: the data are visualized, a baseline model is trained, a stronger transfer-learning model is compared against it, and several metrics are examined. The comparison illustrates the value of transfer learning when the dataset is small, but the outcome should be judged from the measured test results and not assumed in advance.

The main limitations are the small and imbalanced dataset, the controlled image backgrounds, the limited number of training epochs chosen to suit Google Colab, the use of a single data split, and the fact that each image contains a single item. The system classifies an image only; it does not physically sort waste.

4.7 Conclusion

Waste Image Classification implements an end-to-end deep learning workflow in Google Colab. It loads the TrashNet dataset, preprocesses and augments the images, trains a baseline CNN from scratch, trains and fine-tunes a MobileNetV2 transfer-learning model, and evaluates both on unseen data using accuracy, precision, recall, F1-score and confusion matrices. The trained model predicts the category and confidence of a new waste image, and both models are saved in .keras format.

Future improvements include collecting more diverse real-world images with cluttered backgrounds, balancing the classes, trying larger architectures and longer fine-tuning, repeating experiments with cross-validation, adding multi-object detection, deploying the model as a web or mobile application and using real-time camera classification for smart-bin systems.

REFERENCES

1. Yang, M., and Thung, G. (2016). Classification of Trash for Recyclability Status. Stanford University CS229 Project Report (TrashNet dataset). https://github.com/garythung/trashnet

2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

3. LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep Learning. Nature, 521, 436–444.

4. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

5. Kingma, D. P., and Ba, J. (2015). Adam: A Method for Stochastic Optimization. International Conference on Learning Representations (ICLR).

6. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, 15, 1929–1958.

7. TensorFlow. Transfer learning and fine-tuning tutorial. https://www.tensorflow.org/tutorials/images/transfer_learning

8. Keras / TensorFlow documentation. https://www.tensorflow.org/api_docs